

Module 3: Introduction to Computer Graphics and Visual Computing

20 Guided Examples • 20 Practice Exercises

Course: CS0045 – Computer Graphics and Visual Computing

Companion: Module 3 Student Guide

Toolchain: VS Code + MSYS2 UCRT64 GCC + FreeGLUT

How to Use This Manual

This manual has three parts:

- Part A — 20 worked Examples (Ex01–Ex20). Each is a complete, compiled, ready-to-run program demonstrating one concept at a time: bitmap and stroke text, window configuration, and every major GLUT interaction callback. Type or copy each one into your SOURCE CODE\M3 folder, compile it (Ctrl+Shift+B), and run it (F5) before moving to the next.
- Part B — 20 Practice Exercises (Q01–Q20), split into Easy (Q01–Q07), Medium (Q08–Q14), and Hard (Q15–Q20). Read the task and hints, then write the program YOURSELF before checking Part C.
- Part C — the full Answer Key for every exercise in Part B, with complete, compiled, working code.

Setup reminder

Use the shared SOURCE CODE\ VS Code workspace described in the Module 1 Student Guide. Compile with Ctrl+Shift+B ("Build OpenGL Program"), then run with F5 or from the terminal. All programs in this manual use the header block below — note the added freeglut_ext.h include, required for glutBitmapString on nonApple platforms.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#include <GL/freeglut_ext.h>
#endif
#include <iostream> using
namespace std;
```

Every program in this manual has been compiled and verified with g++ against FreeGLUT.

Part A — Guided Examples (Ex01–Ex20)

Example 01 — Basic Bitmap Text

Concept: `glRasterPos2f` + `glutBitmapString(GLUT_BITMAP_HELVETICA_18, ...)` to draw a simple string.

Expected output: The text "Hello, OpenGL!" drawn in white near the center of the window.

```

#define GL_SILENCE_DEPRECATED
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#include <GL/freeglut_ext.h>
#endif
#include <iostream> using
namespace std;

// Helper function to render strings character-by-character
void drawBitmapString(void* font, const char* str) {
    for (const char* c = str; *c != '\0'; c++) {
        glutBitmapCharacter(font, *c);
    }
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    glColor3f(1.0f, 1.0f, 1.0f);    glRasterPos2f(-
0.4f, 0.0f);

    // Draw string using standard GLUT function
    drawBitmapString(GLUT_BITMAP_HELVETICA_18, "Hello,
OpenGL!");

```

```

        glFlush();
    }
int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(600, 300);
    glutCreateWindow("Ex01 - Basic Bitmap Text");
    glutDisplayFunc(display);      glutMainLoop();
    return 0;
}

```

Example 02 — GLUT Bitmap Font Comparison

Concept: Stacking the same string in several different GLUT bitmap font constants.

Expected output: The word "Compare Fonts" repeated multiple times, each in a visibly different font/size.

```

#define GL_SILENCE_DEPRECATED
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#include <GL/freeglut_ext.h>
#endif
#include <iostream> using
namespace std;

/*
 * Example 02 - Comparing GLUT Bitmap Font Constants
 * -----
 * Concept: GLUT provides several fixed-size bitmap fonts.
 * This example

```



* stacks the same word in four different fonts so students

```

can compare
    * their size and style directly.
    */

// Portable helper function to render string character-
bycharacter void drawBitmapString(void* font, const char*
str) {    for (const char* c = str; *c != '\0'; c++) {
glutBitmapCharacter(font, *c);
    }
}

void display() {
glClear(GL_COLOR_BUFFER_BIT);
glColor3f(1.0f, 1.0f, 1.0f);

    glRasterPos2f(-0.8f, 0.6f);
drawBitmapString(GLUT_BITMAP_HELVETICA_10, "Helvetica 10 -
FEU TECH");

    glRasterPos2f(-0.8f, 0.3f);
drawBitmapString(GLUT_BITMAP_HELVETICA_18, "Helvetica 18 -
FEU TECH");

    glRasterPos2f(-0.8f, 0.0f);
drawBitmapString(GLUT_BITMAP_TIMES_ROMAN_10, "Times Roman
10 - FEU TECH");

    glRasterPos2f(-0.8f, -0.3f);
drawBitmapString(GLUT_BITMAP_TIMES_ROMAN_24, "Times
Roman 24 - FEU TECH");

```

```

        glRasterPos2f(-0.8f,                                -0.6f);
drawBitmapString(GLUT_BITMAP_9_BY_15,  "9  by  15  -  FEU
TECH");
        glFlush();
}
int main(int argc, char** argv) {      glutInit(&argc,
argv);      glutInitWindowSize(700, 500);
glutCreateWindow("Ex02 - GLUT Bitmap Font Comparison");
glutDisplayFunc(display);      glutMainLoop();      return 0;
}

```

Example 03 — Text Color — Set BEFORE Raster Position

Concept: Demonstrates that `glColor3f` must be called before `glRasterPos2f` for text to take that color.

Expected output: A red warning message and a green success message, each correctly colored.

```
#define GL_SILENCE_DEPRECATED
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#include <GL/freeglut_ext.h>
#endif
#include <iostream> using
namespace std;

/*
 * Example 03 - Text Color * -----
-----
```



* Concept: glColor3f affects text exactly like it affects

```

any other
*    primitive. IMPORTANT: set the color BEFORE calling
glRasterPos2f, as
*    shown on the lecture slides ("Note: Set the color
before the position").
*/

// Portable helper function using standard GLUT
glutBitmapCharacter void drawBitmapString(void*
font, const char* str) {    for (const char* c =
str; *c != '\0'; c++) {
glutBitmapCharacter(font, *c);
    }
}

void display() {
glClear(GL_COLOR_BUFFER_BIT);

    glColor3f(1.0f, 0.2f, 0.2f);        // red
glRasterPos2f(-0.6f, 0.3f);
drawBitmapString(GLUT_BITMAP_HELVETICA_18, "Warning
Message");

    glColor3f(0.2f, 1.0f, 0.4f);        // green
glRasterPos2f(-0.6f, -0.1f);
drawBitmapString(GLUT_BITMAP_HELVETICA_18, "Success
Message");

glFlush();

```

```

}

int main(int argc, char** argv) {
    glutInit(&argc, argv);      glutInitWindowSize(600,
400);      glutCreateWindow("Ex03 - Text Color");
    glutDisplayFunc(display);    glutMainLoop();
    return 0;
}

```

Example 04 — Multiple Labels at Different Positions

Concept: Several independent `glRasterPos2f + glutBitmapString` pairs in one `display()` call.

Expected output: Five short text labels placed at five different screen locations.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#include <GL/freeglut_ext.h>
#endif
#include <iostream> using
namespace std;

/*
 *   Example 04 - Multiple Text Labels at Different
Positions
 *   -----
 *   Concept: Each glRasterPos2f + drawBitmapString pair is
independent -
 *   you can place as many labels as you like anywhere in
the window.
 */

```

```

// Helper function replacing non-standard glutBitmapString
with glutBitmapCharacter void drawBitmapString(void* font,
const char* str) {      for (const char* c = str; *c !=
'\0'; c++) {            glutBitmapCharacter(font, *c);
    }
}

void display() {
glClear(GL_COLOR_BUFFER_BIT);
glColor3f(1.0f, 1.0f, 1.0f);

    glRasterPos2f(-0.9f, 0.8f);
drawBitmapString(GLUT_BITMAP_HELVETICA_12, "Top-Left");

    glRasterPos2f(0.5f, 0.8f);
drawBitmapString(GLUT_BITMAP_HELVETICA_12, "Top-
Right");

    glRasterPos2f(-0.9f,                -0.9f);
drawBitmapString(GLUT_BITMAP_HELVETICA_12,
"BottomLeft");
    glRasterPos2f(0.5f, -0.9f);
drawBitmapString(GLUT_BITMAP_HELVETICA_12, "BottomRight");

    glRasterPos2f(-0.35f, 0.0f);
drawBitmapString(GLUT_BITMAP_HELVETICA_18, "Center");

```

```
        glFlush();
    }
    int main(int argc, char** argv) {
        glutInit(&argc, argv);        glutInitWindowSize(700,
        500);        glutCreateWindow("Ex04 - Multiple Text
        Labels");        glutDisplayFunc(display);
        glutMainLoop();        return 0;
    }
```

Example 05 — GLUT Stroke (Vector) Font Text

Concept: glutStrokeCharacter looped over a string, combined with glTranslatef/glScalef.

Expected output: The letters "HI" drawn as scalable line-segment (stroke) geometry.

```

#define GL_SILENCE_DEPRECATED
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#include <GL/freeglut_ext.h>
#endif
#include <iostream> using
namespace std;

/*
 *   Example 05 - Stroke Font Text
 *   -----
 *   Concept: Unlike bitmap fonts (fixed-size pixel
stamps), GLUT_STROKE_ROMAN

```



```

*   is a vector font drawn with line segments via
glutStrokeCharacter. Because
*   it is made of real geometry, it can be scaled and
transformed like any
*   other primitive using glScalef/glTranslatef - useful
for zoomable or 3D text.
*/
void display() {
glClear(GL_COLOR_BUFFER_BIT);
glColor3f(0.3f, 0.9f, 1.0f);

    // Optional: Set line width so stroke characters are
clearly visible    glLineWidth(2.0f);

    glPushMatrix();          glTranslatef(-0.8f, -0.1f,
0.0f);          glScalef(0.0015f, 0.0015f, 1.0f); //
stroke fonts are defined in large units; scale down
const char* msg = "STROKE FONT";          for (const char*
c = msg; *c != '\0'; c++) {
glutStrokeCharacter(GLUT_STROKE_ROMAN, *c);
    }
    glPopMatrix();

    glFlush();
}
int main(int argc, char** argv) {
glutInit(&argc, argv);
glutInitWindowSize(600, 300);

```

```
        glutCreateWindow("Ex05 - Stroke Font (Vector) Text");  
glutDisplayFunc(display);    glutMainLoop();    return  
0;  
}
```

Example 06 — Window Size and Position

Concept: `glutInitWindowSize(1024,768)` and `glutInitWindowPosition(400,100)` before `glutCreateWindow`.

Expected output: A 1024x768 window opening at screen position (400, 100), with a caption describing itself.

```

#define GL_SILENCE_DEPRECATED
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#include <GL/freeglut_ext.h>
#endif
#include <iostream> using
namespace std;

/*
 *   Example 06 - Window Initialization and Configuration
 *   -----
 *   Concept: glutInitWindowSize and glutInitWindowPosition
configure the
 *   window BEFORE glutCreateWindow actually creates it -
this is the
 *   "Key Components" sequence from the lecture slides.
 */

// Portable helper function replacing FreeGLUT's

```

```
glutBitmapString void drawBitmapString(void* font,  
const char* str) {     for (const char* c = str; *c
```

```

!= '\0'; c++) {          glutBitmapCharacter(font,
*c);
    }
}

void display() {          glClearColor(0.95f,
0.95f, 0.95f, 1.0f);
glClear(GL_COLOR_BUFFER_BIT);

    glColor3f(0.0f, 0.0f, 0.0f);          glRasterPos2f(-0.6f,
0.0f);          drawBitmapString(GLUT_BITMAP_HELVETICA_18,
"1024 x 768 window at (400, 100)");

    glFlush();
}

int main(int argc, char** argv) {
glutInit(&argc, argv);
glutInitWindowSize(1024, 768);
glutInitWindowPosition(400, 100);
glutCreateWindow("Ex06 - Window Init and
Configuration");
    glutDisplayFunc(display);
glutMainLoop();          return 0;
}

```

Example 07 — Toggling Text with glutPostRedisplay

Concept: SPACE key toggles a boolean and calls glutPostRedisplay() to force a redraw.



Expected output: Text that appears and disappears each time SPACE is pressed.

```

#define GL_SILENCE_DEPRECATED
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#include <GL/freeglut_ext.h>
#endif
#include <iostream> using
namespace std;

/*
 *   Example 07 - glutPostRedisplay Basics
 *   -----
 *   Concept: glutPostRedisplay marks the current window as
needing to be
 *   redrawn. GLUT will call display() again soon after.
Pressing SPACE
 *   toggles a message and requests a redraw.
 */
bool toggled = false;

// Portable helper function replacing FreeGLUT's
glutBitmapString void drawBitmapString(void* font,
const char* str) {   for (const char* c = str; *c
!= '\0'; c++) {      glutBitmapCharacter(font,
*c);
    }
}

```



```
void display() {
```

```

    glClear(GL_COLOR_BUFFER_BIT);
glColor3f(1.0f, 1.0f, 1.0f);    glRasterPos2f(-
0.5f, 0.0f);

    if (toggled) {
drawBitmapString(GLUT_BITMAP_HELVETICA_18, "Toggled
ON");    }
else {
    drawBitmapString(GLUT_BITMAP_HELVETICA_18, "Toggled
OFF");
    }
    glFlush();
}

void keyboard(unsigned char key, int x, int y) {
if (key == ' ') {    toggled = !toggled;
glutPostRedisplay();    // ask GLUT to call
display() again
    }
    if (key == 27) exit(0);
}

int main(int argc, char** argv) {    glutInit(&argc,
argv);    glutInitWindowSize(600, 300);
glutCreateWindow("Ex07 - glutPostRedisplay (press

```

```
SPACE)"); glutDisplayFunc(display);  
    glutKeyboardFunc(keyboard);  
    glutMainLoop();  
  
    return 0;  
}
```

Example 08 — Keyboard-Controlled Text Position

Concept: glutKeyboardFunc: 'a'/'d' move a label left/right; any other key resets its position.

Expected output: A text label that slides horizontally in response to key presses.

```

#define GL_SILENCE_DEPRECATED
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#include <GL/freeglut_ext.h>
#endif
#include <iostream> using
namespace std;

/*
 *   Example 08 - glutKeyboardFunc: Move Text with 'a' /
 *   'd'
 *   -----
 *   Concept: Mirrors the lecture's keyboardMonitor()
 *   sample - pressing 'a'
 *   moves the label left, 'd' moves it right, and any
 *   other key resets it.
 */
float px = 0.0f, py = 0.0f;

// Portable helper function replacing FreeGLUT's
glutBitmapString void drawBitmapString(void* font,
const char* str) {
    for (const char* c = str; *c != '\0'; c++) {

```

```

        glutBitmapCharacter(font, *c);
    }
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.0f, 0.4f, 1.0f);
    glRasterPos2f(px, py);
    drawBitmapString(GLUT_BITMAP_HELVETICA_18, "FEU-TECH");
    glFlush();
}

void keyboardMonitor(unsigned char key, int x, int y) {
    cout << key << endl;
    switch (key) {
        case 'a':
            px -= 0.02f;
            break;
        case 'd':
            px += 0.02f;
            break;
        default:
            px = 0.0f;
    }
    glutPostRedisplay();
}

int main(int argc, char** argv) {

```

```
glutInit(&argc, argv);  
glutInitWindowSize(1024, 768);  
glutInitWindowPosition(200, 50);  
glutCreateWindow("Ex08 - Move Text (a = left,  
d =
```

```
right)");    glutDisplayFunc(display);  
glutKeyboardFunc(keyboardMonitor);  
glutMainLoop();    return 0;  
}
```

Example 09 — Mouse Button Detection

Concept: glutMouseFunc changes a square's fill color depending on which button was clicked.

Expected output: A square that changes color on left-click vs. right-click.

```

#define GL_SILENCE_DEPRECATED
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#include <GL/freeglut_ext.h>
#endif
#include <iostream> using
namespace std;

/*
 * Example 09 - glutMouseFunc: Log Clicks and Change Color
 * -----
 * Concept: glutMouseFunc's callback fires on every press
AND release.
 * button is one of
GLUT_LEFT_BUTTON/GLUT_MIDDLE_BUTTON/GLUT_RIGHT_BUTTON;
 * state is either GLUT_DOWN or GLUT_UP.
 */

float sqColor[3] = {1.0f, 1.0f, 1.0f};

```

```

void display() {      glClearColor(GL_COLOR_BUFFER_BIT);
glColor3f(sqColor[0], sqColor[1], sqColor[2]);
glBegin(GL_POLYGON);      glVertex2f(-0.3f, -
0.3f);      glVertex2f(-0.3f,  0.3f);
glVertex2f( 0.3f,  0.3f);      glVertex2f(
0.3f, -0.3f);      glEnd();      glFlush();
}

void mouseMonitor(int button, int state, int x, int y) {
if (button == GLUT_LEFT_BUTTON && state == GLUT_DOWN) {
cout << "Left button was clicked..." << endl;
sqColor[0] = 1.0f; sqColor[1] = 0.0f; sqColor[2] =
0.0f;  // red
    } else if (button == GLUT_RIGHT_BUTTON && state ==
GLUT_DOWN) {
        cout << "Right button was clicked..." << endl;
sqColor[0] = 0.0f; sqColor[1] = 0.4f; sqColor[2] =
1.0f;  // blue
    } else if (button == GLUT_MIDDLE_BUTTON && state ==
GLUT_DOWN) {
        cout << "Middle button was clicked..." << endl;
sqColor[0] = 0.0f; sqColor[1] = 1.0f; sqColor[2] = 0.0f;
// green
    }
}

```

```

        cout << x << " " << y << endl;
        glutPostRedisplay();
    }

int main(int argc, char** argv) {    glutInit(&argc,
argv);    glutInitWindowSize(600, 600);
glutCreateWindow("Ex09 - Mouse Clicks Change Square
Color");
    glutDisplayFunc(display);
glutMouseFunc(mouseMonitor);
glutMainLoop();    return 0;
}

```

Example 10 — Active Motion — Drawing While Dragging

Concept: glutMotionFunc converts pixel coordinates to OpenGL coordinates and plots a point at the drag position.

Expected output: A trail of points that appears only while the mouse button is held and dragged.

```
#define GL_SILENCE_DEPRECATED
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#include <GL/freeglut_ext.h>
#endif
#include <iostream> using
namespace std;

/*
 * Example 10 - glutMotionFunc: Active Motion (Dragging)
 * -----
 * Concept: The active motion callback fires only while a
```

```

mouse button IS
*   held down and the mouse moves. This example draws a
point wherever the
*   mouse is dragged, converting pixel coordinates to
OpenGL's -1..1 range.
*/
float lastX = 0.0f, lastY = 0.0f; bool
hasPoint = false;

void toOpenGLCoords(int x, int y, float &outX, float &outY)
{
    int w = glutGet(GLUT_WINDOW_WIDTH);    int h =
glutGet(GLUT_WINDOW_HEIGHT);    outX = (2.0f * x /
w) - 1.0f;    outY = 1.0f - (2.0f * y / h);    //
pixel Y grows downward; OpenGL Y grows upward
}

void display() {
glClear(GL_COLOR_BUFFER_BIT);    if
(hasPoint) {        glColor3f(1.0f,
1.0f, 0.0f);
glPointSize(10.0f);
glBegin(GL_POINTS);
glVertex2f(lastX, lastY);
glEnd();
    }
}

```

```

        glFlush();
    }

void detectMotion(int x, int y) {
    toOpenGLCoords(x, y, lastX, lastY);
    hasPoint = true;      cout << x << " "
<< y << endl;          glutPostRedisplay();
}

int main(int argc, char** argv) {      glutInit(&argc,
argv);      glutInitWindowSize(600, 600);
glutCreateWindow("Ex10 - Drag the Mouse to Place a
Point");
    glutDisplayFunc(display);
glutMotionFunc(detectMotion);
glutMainLoop();      return 0;
}

```

Example 11 — Passive Motion — Live Coordinate Readout

Concept: `glutPassiveMotionFunc` displays "Mouse at (x, y)" as bitmap text, updating with no button held.

Expected output: A text readout that follows the mouse position as it moves over the window.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else

```

```
#include <GL/glut.h>
#include <GL/freeglut_ext.h>
#endif
#include <cstdio>
#include <iostream>
```



```
using namespace std;
```



```

/*
 *   Example 11 - glutPassiveMotionFunc: Track Mouse Without
Clicking
 *   -----
 *   Concept: The passive motion callback fires when the
mouse moves within
 *   the window while NO button is pressed - useful for
hover effects and
 *   live coordinate readouts.
 */
int mouseX = 0, mouseY = 0;

// Portable helper function replacing FreeGLUT's
glutBitmapString void drawBitmapString(void* font,
const char* str) {    for (const char* c = str; *c
!= '\0'; c++) {        glutBitmapCharacter(font,
*c);
    }
}

void display() {
glClear(GL_COLOR_BUFFER_BIT);
glColor3f(1.0f, 1.0f, 1.0f);

    char buffer[64];    snprintf(buffer, sizeof(buffer),
"Mouse at (%d, %d)", mouseX, mouseY);    glRasterPos2f(-
0.4f, 0.0f);
drawBitmapString(GLUT_BITMAP_HELVETICA_18, buffer);

```

```

        glFlush();
    }
    void detectPassiveMotion(int x, int y) {
        mouseX = x;        mouseY = y;
        glutPostRedisplay();
    }
    int main(int argc, char** argv) {        glutInit(&argc,
        argv);        glutInitWindowSize(600, 400);
        glutCreateWindow("Ex11 - Passive Motion (Move Mouse, No
        Click)");
        glutDisplayFunc(display);
        glutPassiveMotionFunc(detectPassiveMotion);
        glutMainLoop();        return 0;
    }

```

Example 12 — Entry/Exit Detection

Concept: `glutEntryFunc` changes the background color when the mouse enters vs. leaves the window.

Expected output: The background switches between two colors as the mouse crosses the window boundary.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>

```

```
#else  
#include <GL/glut.h>  
#include <GL/freeglut_ext.h>
```



```
#endif
```

```

#include <iostream> using
namespace std;

/*
 *   Example 12 - glutEntryFunc: Mouse Enter/Leave
 *   -----
 *   Concept: glutEntryFunc's callback fires when the mouse
pointer enters
 *   or leaves the window. state is GLUT_LEFT (the pointer
left the window)
 *   or GLUT_ENTERED (the pointer re-entered it).
 */
float bg[3] = {0.1f, 0.1f, 0.1f};

void display() {      glClearColor(bg[0],
bg[1], bg[2], 1.0f);
glClear(GL_COLOR_BUFFER_BIT);      glFlush();
}

void mouseEntryDetector(int state) {      if (state ==
GLUT_LEFT) {          cout << "Mouse has left the
application " << endl;          bg[0] = 0.5f; bg[1] = 0.1f;
bg[2] = 0.1f; // reddish      } else {
      cout << "Mouse has re-entered the application " <<
endl;          bg[0] = 0.1f; bg[1] = 0.4f; bg[2] = 0.1f;
//

```

greenish

```

    }
    glutPostRedisplay();
}
int main(int argc, char** argv) {
    glutInit(&argc, argv);      glutInitWindowSize(600,
400);      glutCreateWindow("Ex12 - Mouse Enter/Leave
Changes Background");
    glutDisplayFunc(display);
    glutEntryFunc(mouseEntryDetector);
    glutMainLoop();      return 0;
}

```

Example 13 — Idle-Driven Bouncing Label

Concept: `glutIdleFunc` increments a label's X position every frame and reverses direction at the edges.

Expected output: A text label that continuously bounces left and right across the window.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#include <GL/freeglut_ext.h>
#endif
#include <iostream> using
namespace std;
```

```
/*
```

```
* Example 13 - glutIdleFunc: Continuously Animate a Text
```



Label

```

* -----
*   Concept: glutIdleFunc sets the global idle callback,
which GLUT calls
*   repeatedly whenever no other events are pending - the
classic way to
*   drive continuous animation.
*/
float px = -0.8f; float
speed = 0.01f;

// Portable helper function replacing FreeGLUT's
glutBitmapString void drawBitmapString(void* font,
const char* str) {      for (const char* c = str; *c
!= '\0'; c++) {          glutBitmapCharacter(font,
*c);
    }
}

void display() {      glClear(GL_COLOR_BUFFER_BIT);
glColor3f(1.0f, 1.0f, 1.0f);      glRasterPos2f(px,
0.0f);      drawBitmapString(GLUT_BITMAP_HELVETICA_18,
"Sliding Label");      glFlush();
}

```

```

void animateText() {
    px += speed;

    if (px > 0.6f || px < -0.9f) {
        speed = -speed;
    }

    glutPostRedisplay();
}

int main(int argc, char** argv) {    glutInit(&argc,
argv);    glutInitWindowSize(600, 300);
glutCreateWindow("Ex13 - glutIdleFunc Animated Label");
glutDisplayFunc(display);    glutIdleFunc(animateText);
glutMainLoop();    return 0;
}

```

Example 14 — Timer-Driven Shape Movement

Concept: A self-rescheduling `glutTimerFunc(1000, ...)` moves a square once per second.

Expected output: A square that jumps to a new horizontal position once every second.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#include <GL/freeglut_ext.h>
#endif
#include <iostream> using
namespace std;

/*
 * Example 14 - glutTimerFunc: Move a Shape Every Second
 * -----
```

* Concept: `glutTimerFunc(msecs, func, value)` schedules

```

func to be called
*     ONCE after msec milliseconds, passing it value. To
repeat, the timer
*     callback must call glutTimerFunc again itself - this is
called a
*     "self-rescheduling" timer.
*/
float px = -0.6f;

void display() {
glClear(GL_COLOR_BUFFER_BIT);
glColor3f(1.0f, 0.6f, 0.0f);
glBegin(GL_POLYGON);
glVertex2f(px - 0.1f, -0.1f);
glVertex2f(px - 0.1f, 0.1f);
glVertex2f(px + 0.1f, 0.1f);
glVertex2f(px + 0.1f, -0.1f);
glEnd();      glFlush();
}
void animateStep(int value) {
px += 0.1f;    if (px > 0.6f)
px = -0.6f;

    glutPostRedisplay();      glutTimerFunc(1000,
animateStep, value); // reschedule

```

```
itself every 1000ms
}
```

```
int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(600, 400);
    glutCreateWindow("Ex14 - glutTimerFunc Moves a Square
Every Second");
    glutDisplayFunc(display);    glutTimerFunc(1000,
animateStep, 0);    glutMainLoop();    return 0;
}
```

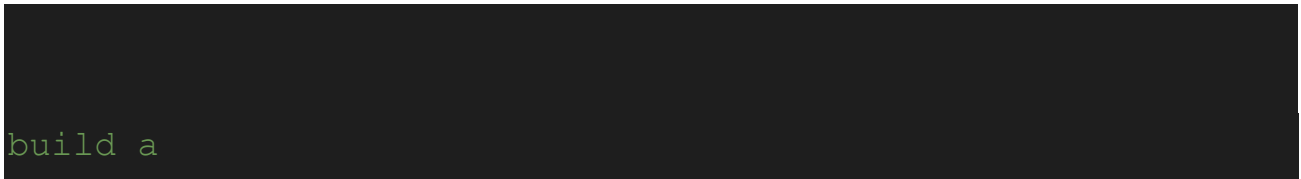
Example 15 — Timer-Driven Countdown Display

Concept: glutTimerFunc counts down from 10, displaying "Time left: N" as text each second.

Expected output: A countdown from 10 to 0 displayed as on-screen text, updating once per second.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#include <GL/freeglut_ext.h>
#endif
#include <cstdio>
#include <iostream> using
namespace std;

/*
* Example 15 - glutTimerFunc: Countdown Timer as Text
* -----
* Concept: Combines glutTimerFunc with drawBitmapString to
```



```

*    simple countdown display, decrementing once per second
until it
*    reaches zero.
*/
int secondsLeft = 10;

// Portable helper function replacing FreeGLUT's
glutBitmapString void drawBitmapString(void* font,
const char* str) {    for (const char* c = str; *c
!= '\0'; c++) {        glutBitmapCharacter(font,
*c);
    }
}

void display() {
glClear(GL_COLOR_BUFFER_BIT);
glColor3f(1.0f, 1.0f, 1.0f);

    char buffer[32];    snprintf(buffer,
sizeof(buffer), "Time left: %d", secondsLeft);
glRasterPos2f(-0.3f, 0.0f);
drawBitmapString(GLUT_BITMAP_HELVETICA_18, buffer);

    glFlush();
}

```



```
void tick(int value) {  
    if (secondsLeft > 0) {
```

```

        secondsLeft--;
glutPostRedisplay();
        glutTimerFunc(1000, tick, 0);
    }
}

int main(int argc, char** argv) {
glutInit(&argc, argv);
glutInitWindowSize(600, 300);
glutCreateWindow("Ex15 - Countdown Timer");
glutDisplayFunc(display);
glutTimerFunc(1000, tick, 0);
glutMainLoop();    return 0;
}

```

Example 16 — Shape Combined with a Caption

Concept: A filled triangle drawn together with a bitmap-text caption describing it.

Expected output: A colored triangle with the caption text positioned just below it.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#include <GL/freeglut_ext.h>
#endif
#include <iostream> using
namespace std;

/*
 * Example 16 - Combining Text and a Shape

```



★



```

*    Concept: A shape and a text label are just two
different sets of
*    drawing calls in the same display() function - order
does not matter
*    for 2D flat rendering like this.
*/

// Portable helper function replacing FreeGLUT's
glutBitmapString void drawBitmapString(void* font,
const char* str) {    for (const char* c = str; *c
!= '\0'; c++) {        glutBitmapCharacter(font,
*c);
    }
}

void display() {
glClear(GL_COLOR_BUFFER_BIT);

    glColor3f(0.2f, 0.6f, 1.0f);
glBegin(GL_TRIANGLES);
glVertex2f(-0.3f,      -0.3f);
glVertex2f(    0.3f,   -0.3f);
glVertex2f(    0.0f,    0.3f);
glEnd();

    glColor3f(1.0f, 1.0f, 1.0f);    glRasterPos2f(-0.35f,
-0.6f);    drawBitmapString(GLUT_BITMAP_HELVETICA_18,
"Figure 1: A Triangle");

glFlush(); }

```

```
int main(int argc, char** argv) {      glutInit(&argc,
argv);      glutInitWindowSize(600, 500);
glutCreateWindow("Ex16 - Shape with a Caption");
glutDisplayFunc(display);      glutMainLoop();
return 0;
}
```

Example 17 — Toggling Full-Screen Mode

Concept: A mouse click calls `glutFullScreen()`; a second click restores the window with `glutReshapeWindow`.

Expected output: The window expands to fill the screen on click, and shrinks back on the next click.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#include <GL/freeglut_ext.h>
#endif
#include <iostream> using
namespace std;

/*
 *   Example 17 - glutFullScreen: Toggle Fullscreen on
 *   Click
 *   -----
 *   Concept: glutFullScreen() requests that the CURRENT
 *   window be made
 *   full screen. There is no built-in "un-fullscreen" call
 *   in classic
```

```

restore the
    * original size.
    */
bool isFullscreen = false;

// Portable helper function replacing FreeGLUT's
glutBitmapString void drawBitmapString(void* font,
const char* str) {      for (const char* c = str; *c
!= '\0'; c++) {          glutBitmapCharacter(font,
*c);
    }
}

void display() {      glClear(GL_COLOR_BUFFER_BIT);
glColor3f(1.0f, 1.0f, 1.0f);      glRasterPos2f(-0.5f,
0.0f);      drawBitmapString(GLUT_BITMAP_HELVETICA_18,
"Click to toggle fullscreen");      glFlush();
}

void mouseClicked(int button, int state, int x, int y) {
if (button == GLUT_LEFT_BUTTON && state == GLUT_DOWN) {
if (!isFullscreen) {          glutFullScreen();
    } else {
        glutReshapeWindow(600, 400);
    }
    isFullscreen = !isFullscreen;
}
}

```

```

* GLUT, so this example uses glutReshapeWindow to manually
}
}
int main(int argc, char** argv) {      glutInit(&argc,
argv);      glutInitWindowSize(600, 400);
glutCreateWindow("Ex17 - glutFullScreen (click to
toggle)");      glutDisplayFunc(display);
glutMouseFunc(mouseClick);      glutMainLoop();
return 0;
}

```

Example 18 — Full Callback Integration

Concept: display, keyboard, mouse, motion, passive motion, and entry callbacks all registered together.

Expected output: A single interactive scene responding to keyboard, clicks, dragging, hovering, and window entry/exit.

```
#define GL_SILENCE_DEPRECATED
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#include <GL/freeglut_ext.h>
#endif
#include <iostream> using
namespace std;

/*
 * Example 18 - Full Callback Integration
 * -----
 * Concept: Mirrors the lecture's complete sample program,
```

```

wiring up
*   EVERY major GLUT callback in one application: display,
keyboard,
*   mouse, motion, passive motion, and entry.
*/
float px = 0.0f, py = 0.0f;

// Portable helper function replacing FreeGLUT's
glutBitmapString void drawBitmapString(void* font,
const char* str) {   for (const char* c = str; *c
!= '\0'; c++) {      glutBitmapCharacter(font,
*c);
    }
}

void keyboardMonitor(unsigned char key, int x, int y) {
cout << key << endl;    switch (key) {        case
'a': px -= 0.02f; break;        case 'd': px += 0.02f;
break;        default: px = 0.0f;
    }
    glutPostRedisplay();
}

void mouseMonitor(int button, int state, int x, int y) {
if (button == GLUT_LEFT_BUTTON) {        if (state ==
GLUT_DOWN) cout << "Left button was clicked..." << endl;
else cout << "Left button was released..." << endl;
}
}

```



```
} else if (button == GLUT_RIGHT_BUTTON) {
```

```

        cout << "Right button was clicked..." << endl;
    } else {
        cout << "Middle button was clicked..." << endl;
    }
    cout << x << " " << y << endl;
}

void detectMotion(int x, int y) {
    cout << x << " " << y << endl;
}

void detectPassiveMotion(int x, int y) {
    cout << x << "P " << y << endl;
}

void mouseEntryDetector(int state) {    if (state ==
GLUT_LEFT) cout << "Mouse has left the application " <<
endl;    else cout << "Mouse has re-entered the
application " << endl;
}

void defaultDisplay() {
    glClear(GL_COLOR_BUFFER_BIT);

    glColor3f(0.0f, 0.0f, 1.0f);    glRasterPos2f(px, py);
drawBitmapString(GLUT_BITMAP_HELVETICA_18, "FEU-TECH");

    glFlush();
}

```



```

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(1024, 768);
    glutInitWindowPosition(200, 50);

    glutCreateWindow("Ex18 - Full Callback Integration");
    glutDisplayFunc(defaultDisplay);
    glutKeyboardFunc(keyboardMonitor);
    glutMouseFunc(mouseMonitor);
    glutMotionFunc(detectMotion);
    glutPassiveMotionFunc(detectPassiveMotion);
    glutEntryFunc(mouseEntryDetector);    glutMainLoop();

    return 0;
}

```

Example 19 — HUD Score Overlay

Concept: Each mouse click increments a score variable, displayed as HUD-style text over a shape.

Expected output: A shape with a "Score: N" counter in the corner that increases with each click.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#include <GL/freeglut_ext.h>
#endif
#include <cstdio>
#include <iostream> using
namespace std;
```

```
/*
```

```
* Example 19 - HUD-Style Overlay
```

```
* -----
```

```

*    Concept: Many games overlay 2D text (a "HUD" - heads-up
display) on
*    top of a scene. Here, clicking increases a score
counter shown as
*    bitmap text above a simple shape "scene".
*/
int score = 0;

// Portable helper function replacing FreeGLUT's
glutBitmapString void drawBitmapString(void* font,
const char* str) {    for (const char* c = str; *c
!= '\0'; c++) {        glutBitmapCharacter(font,
*c);
    }
}

void display() {
glClear(GL_COLOR_BUFFER_BIT);

    // "scene"
    glColor3f(0.2f, 0.7f, 0.3f);
glBegin(GL_POLYGON);        glVertex2f(-
0.3f, -0.3f);        glVertex2f(-0.3f,
0.3f);        glVertex2f( 0.3f,  0.3f);
glVertex2f( 0.3f, -0.3f);    glEnd();

```

```

        // HUD text
char buffer[32];
    snprintf(buffer, sizeof(buffer), "Score: %d", score);
    glColor3f(1.0f, 1.0f, 1.0f);    glRasterPos2f(-
0.9f, 0.85f);
drawBitmapString(GLUT_BITMAP_HELVETICA_18, buffer);

    glFlush();
}
void onClick(int button, int state, int x, int y) {    if
(button == GLUT_LEFT_BUTTON && state == GLUT_DOWN) {
score += 10;
    glutPostRedisplay();
}
}
int main(int argc, char** argv) {
glutInit(&argc, argv);    glutInitWindowSize(600,
600);    glutCreateWindow("Ex19 - HUD Overlay
(click to score)");    glutDisplayFunc(display);
glutMouseFunc(onClick);    glutMainLoop();
return 0;
}

```



Example 20 — Capstone: Interactive Text/Marker Placer

Concept: Clicking anywhere converts pixel coordinates to OpenGL coordinates and repositions a shape + label there; idle-driven pulsing and a live passive-motion readout are layered on top.

Expected output: Clicking anywhere in the window moves a pulsing marker and its label to that exact point, while a live coordinate readout tracks the mouse.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#include <GL/freeglut_ext.h>
#endif
#include <iostream> using
namespace std;

/*
 *   Example 20 - Capstone: Interactive Text Placer
 *   -----
 *   Concept: Combines glutMouseFunc, pixel-to-OpenGL
coordinate conversion
 *   (using glutGet(GLUT_WINDOW_WIDTH/HEIGHT)), and
glRasterPos2f: clicking
 *   anywhere in the window moves the text label to that
exact spot.
 */
float labelX = -0.3f, labelY = 0.0f;

// Portable helper function replacing FreeGLUT's
glutBitmapString void drawBitmapString(void* font,
const char* str) {   for (const char* c = str; *c
!= '\0'; c++) {      glutBitmapCharacter(font,
*c);   } }
```

```

}
void toOpenGLCoords(int x, int y, float &outX, float &outY)
{
    int w = glutGet(GLUT_WINDOW_WIDTH);
    int h = glutGet(GLUT_WINDOW_HEIGHT);
    outX = (2.0f * x / w) - 1.0f;    outY =
    1.0f - (2.0f * y / h);
}
void display() {    glClear(GL_COLOR_BUFFER_BIT);
glColor3f(1.0f, 0.85f, 0.2f);
glRasterPos2f(labelX, labelY);
drawBitmapString(GLUT_BITMAP_HELVETICA_18, "Click
anywhere!");    glFlush();
}
void onClick(int button, int state, int x, int y) {    if
(button == GLUT_LEFT_BUTTON && state == GLUT_DOWN) {
toOpenGLCoords(x, y, labelX, labelY);
glutPostRedisplay();
    }
}
int main(int argc, char** argv) {    glutInit(&argc,
argv);    glutInitWindowSize(700, 500);
glutCreateWindow("Ex20 - Interactive Text Placer");
glutDisplayFunc(display);    glutMouseFunc(onClick);

```

```
glutMainLoop();    return 0;  
}
```

Part B — Practice Exercises (Q01–Q20)

Attempt each exercise on your own first. Compile and run your own solution, compare its output to the task description, and only THEN check Part C. Difficulty is marked Easy, Medium, or Hard.

Easy Exercises

Exercise Q01 — Display Your Name [Easy]

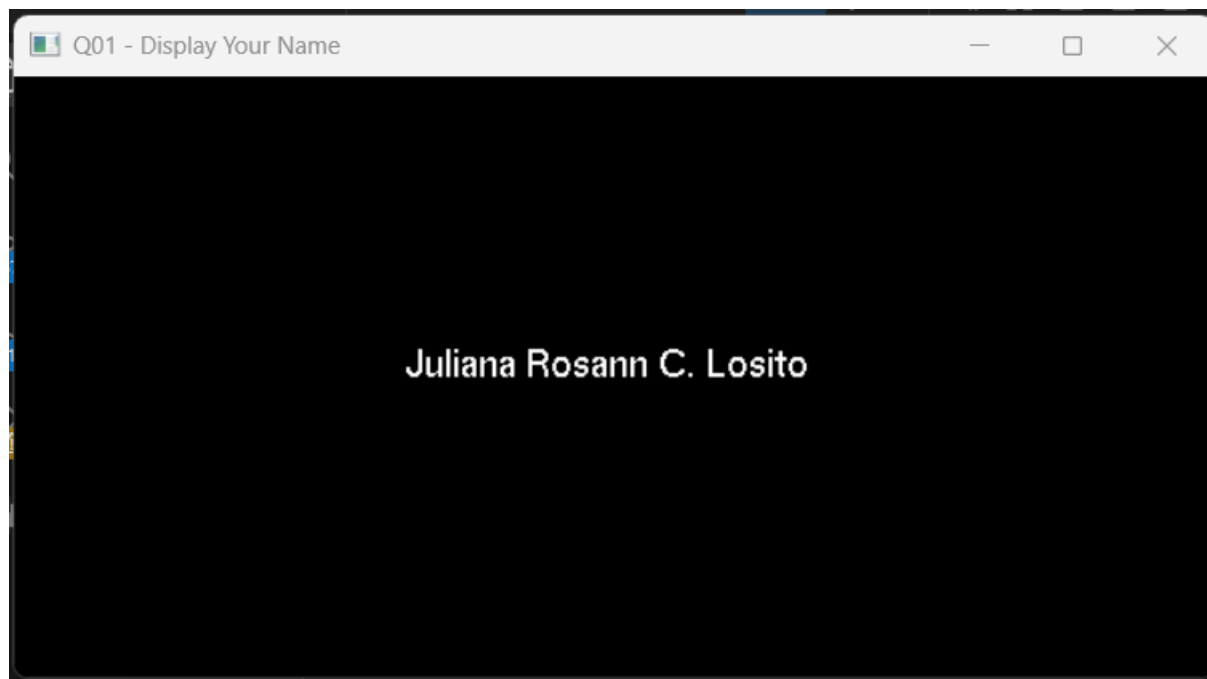
Objective: Practice the basic glColor3f + glRasterPos2f + glutBitmapString workflow.

Task: Display your own name as white bitmap text near the center of the window, using GLUT_BITMAP_HELVETICA_18.

Hints:

- Set color BEFORE calling glRasterPos2f, not after.
- Cast your string to (const unsigned char*) if needed.

Output:



Exercise Q02 — Font Size Comparison [Easy]

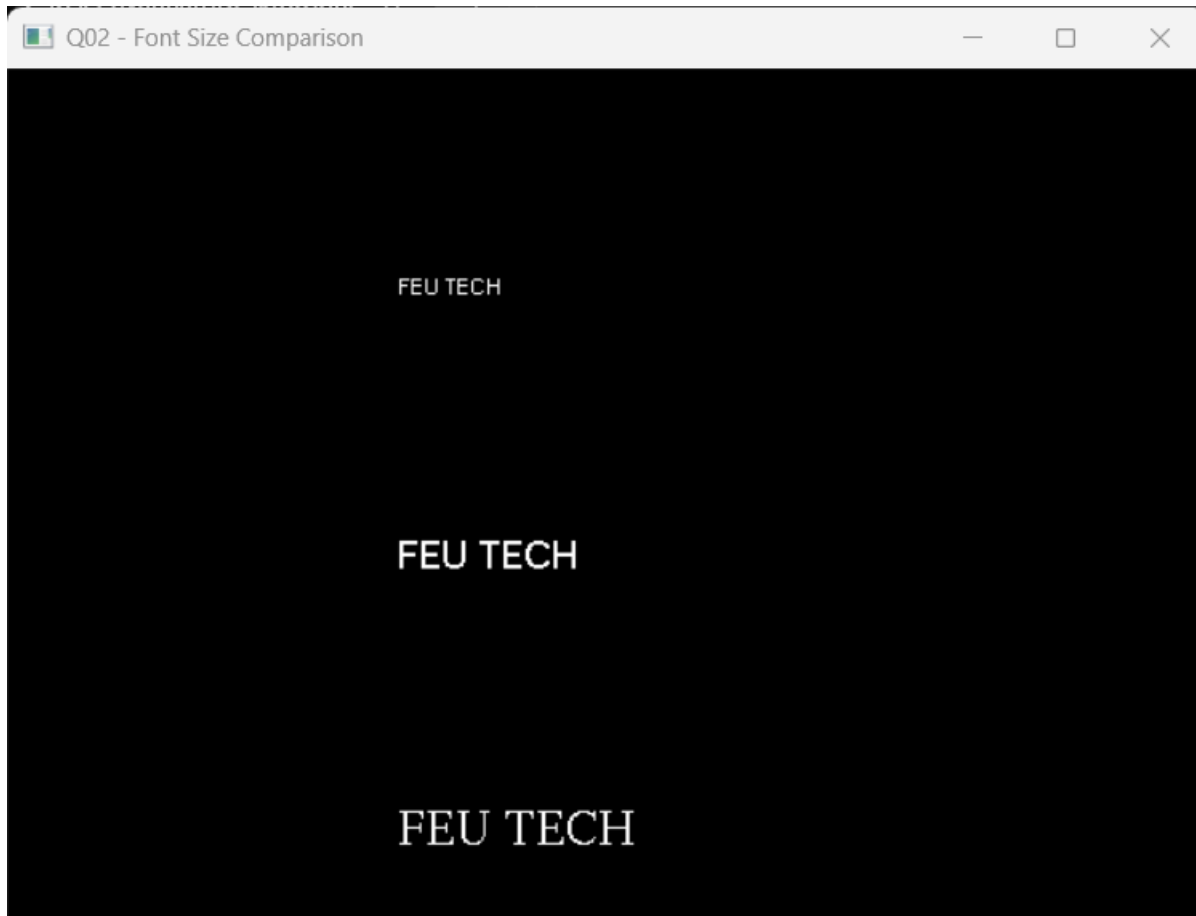
Objective: Practice using multiple GLUT bitmap font constants in one program.

Task: Display the same short phrase three times, stacked vertically, each using a different bitmap font size (e.g. HELVETICA_10, HELVETICA_18, TIMES_ROMAN_24).

Hints:

- Give each line of text a different Y value for `glRasterPos2f` so they don't overlap.
- Leave enough vertical spacing for the largest font.

Output:



Exercise Q03 — Custom Window Setup [Easy]

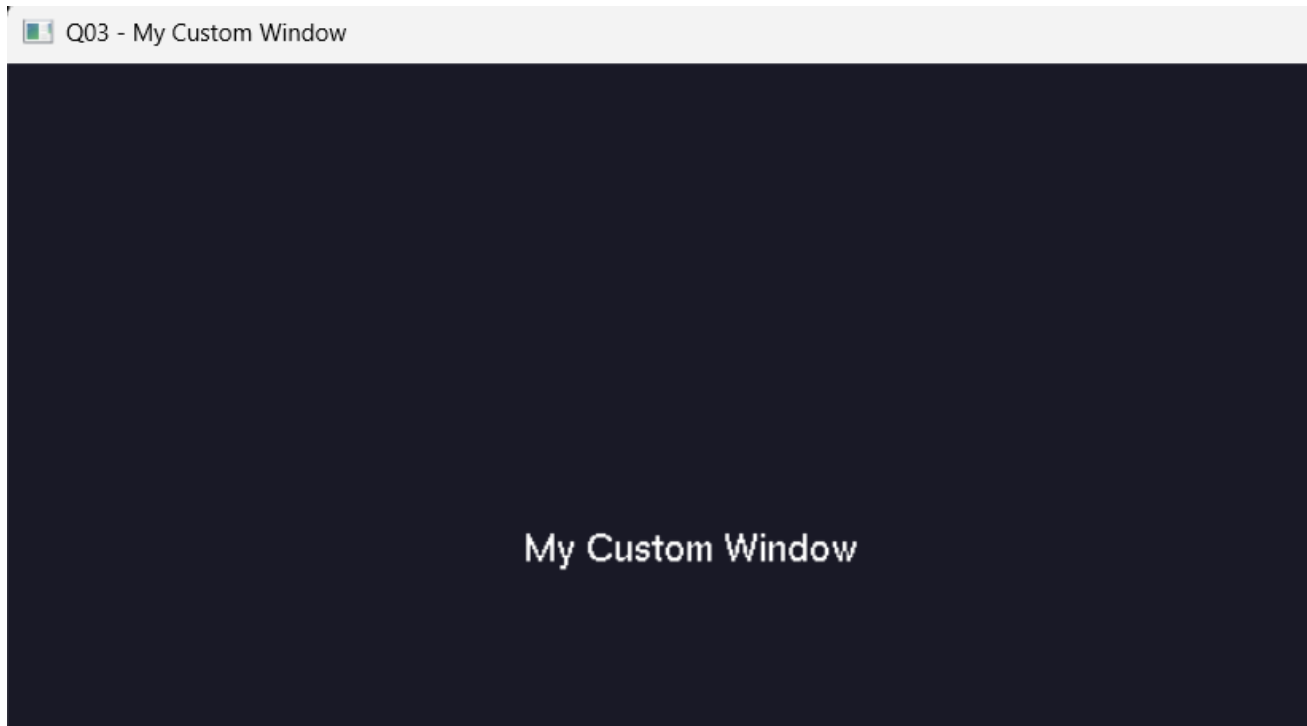
Objective: Practice configuring window size, position, and title before creation.

Task: Create a window sized 800x500 pixels, positioned at (150, 150) on the screen, titled "My Custom Window".

Hints:

- `glutInitWindowSize` and `glutInitWindowPosition` must be called BEFORE `glutCreateWindow`.
- The title is passed directly as `glutCreateWindow`'s argument.

Output:



Exercise Q04 — Keyboard Background Color Switch [Easy]

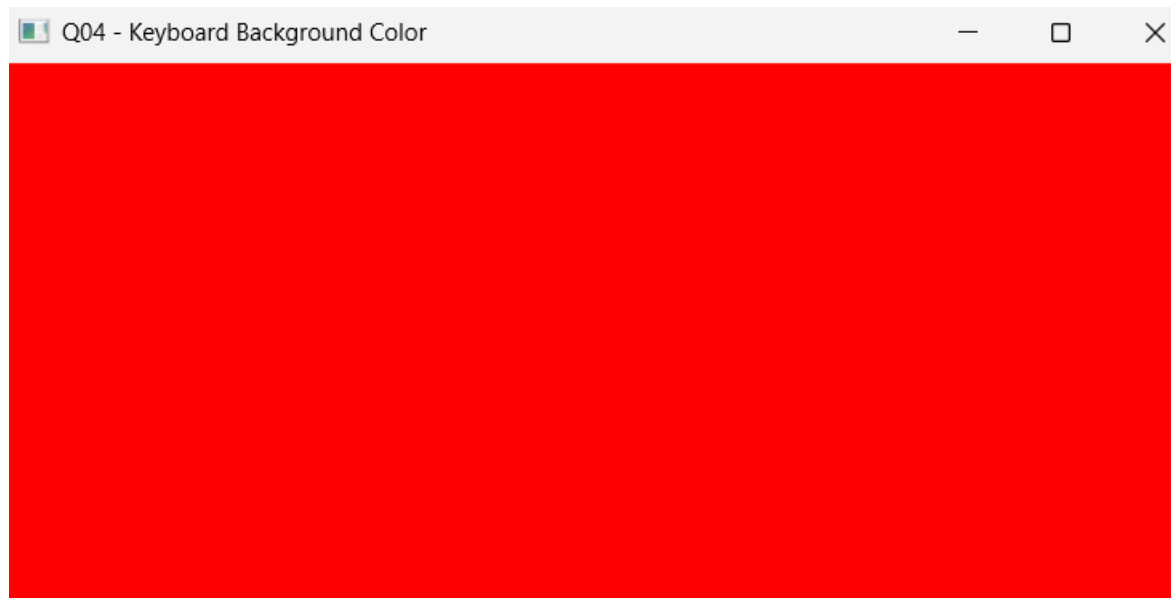
Objective: Practice `glutKeyboardFunc` combined with `glutPostRedisplay`.

Task: Pressing 'r', 'g', or 'b' should change the window's background clear color to red, green, or blue respectively.

Hints:

- Store the background color in global float variables and update them inside your keyboard callback.
- Don't forget `glutPostRedisplay()` after changing the color.

Output:



Exercise Q05 — Mouse Button Text Color [Easy]

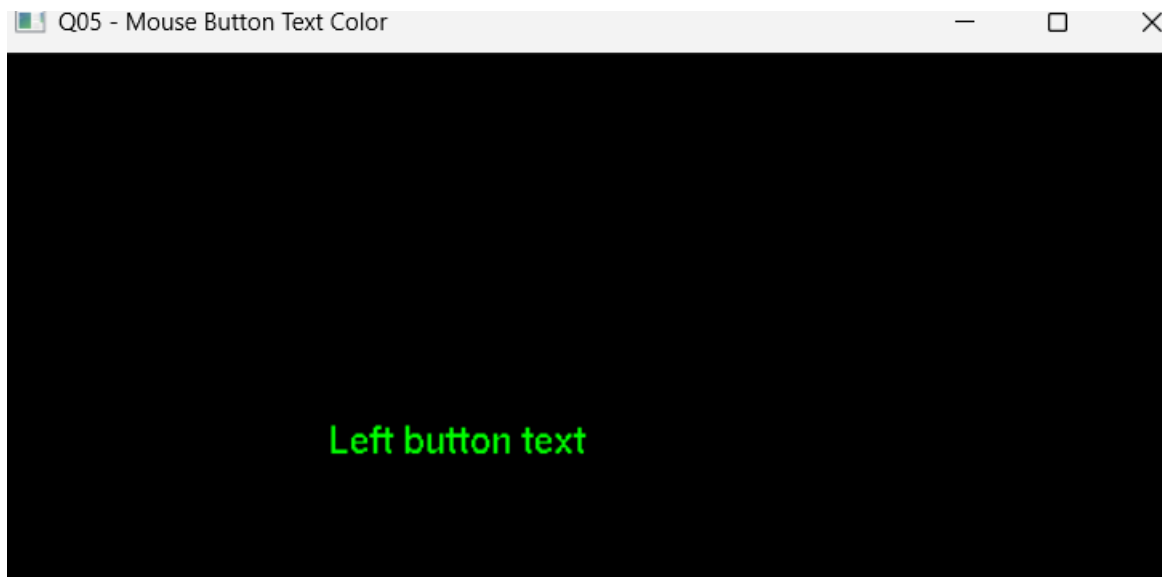
Objective: Practice `glutMouseFunc` combined with conditional bitmap text.

Task: Clicking the left mouse button should display "Left button text" in green; clicking the right button should display "Right button text" in red.

Hints:

- Check for `state == GLUT_DOWN` so the message doesn't flicker on release too.
- Remember to set color before `glRasterPos2f` for each message.

Output:



Exercise Q06 — Shape with Caption [Easy]

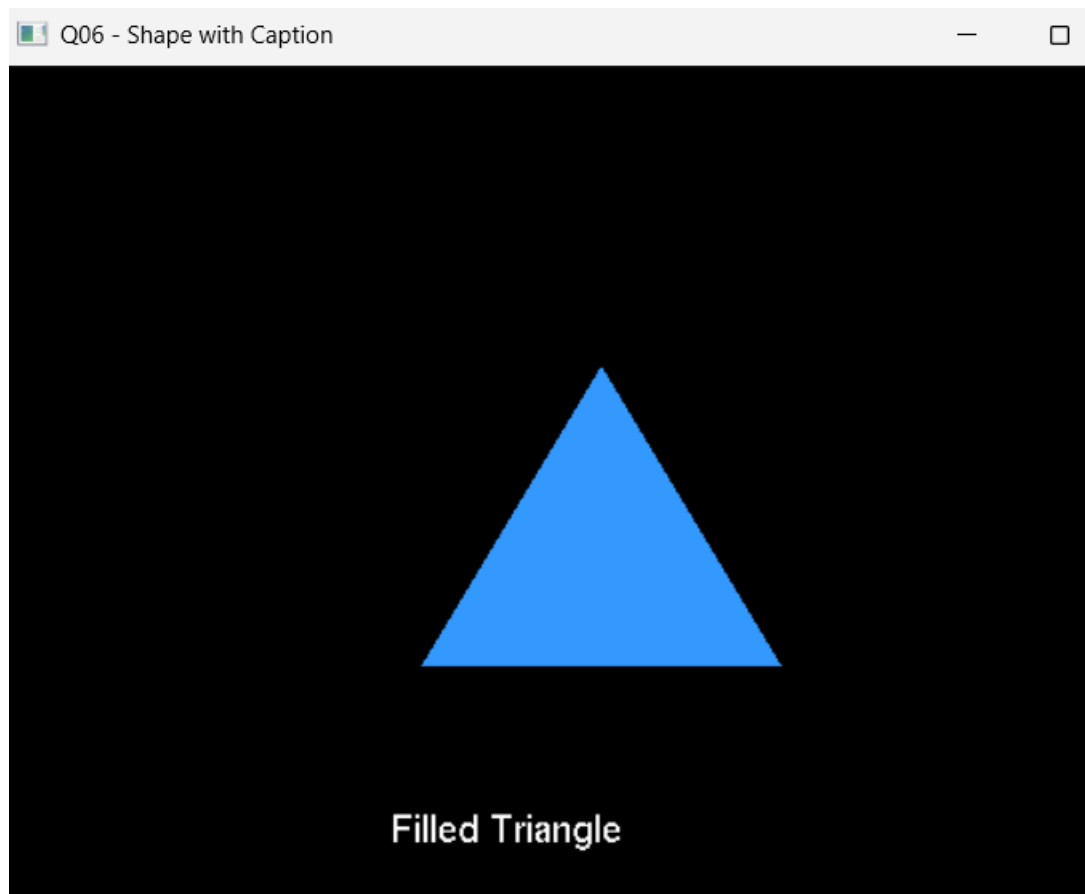
Objective: Practice combining ordinary geometry with a bitmap-text caption.

Task: Draw a filled triangle and place a short caption below it describing the shape, using bitmap text.

Hints:

- Draw the geometry first, then the text — order inside `display()` does not usually matter for 2D, but keep it consistent.
- Position the caption's raster position clearly below the shape's lowest vertex.

Output:



Exercise Q07 — Stroke Font Practice [Easy]

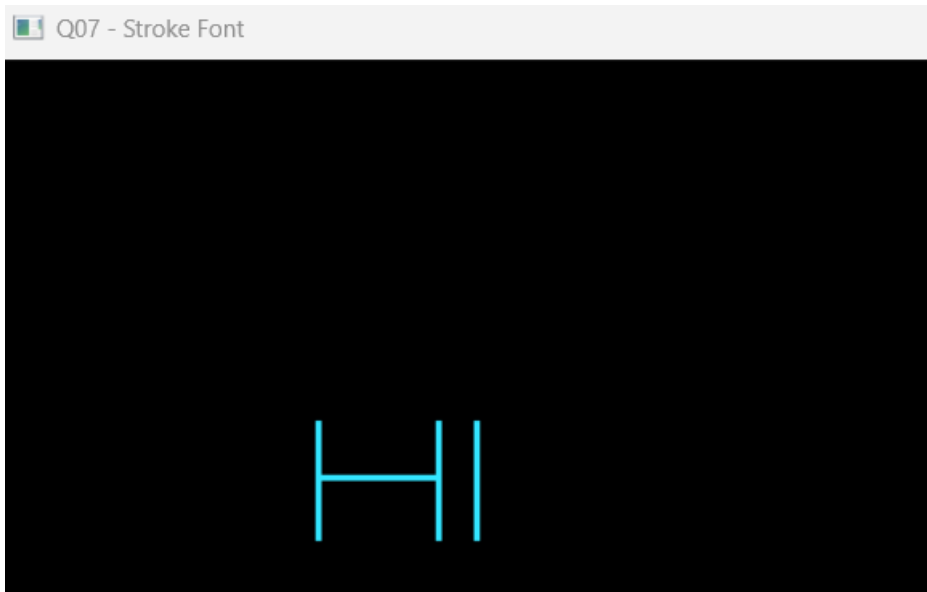
Objective: Practice `glutStrokeCharacter` with `glTranslatef`/`glScalef`.

Task: Draw the letters "HI" using the `GLUT_STROKE_ROMAN` vector font, scaled and positioned so it is clearly visible in the window.

Hints:

- Stroke font coordinates are large internally — try a small scale factor like `0.003f` as a starting point.
- Loop over each character of your string, calling `glutStrokeCharacter` once per character.

Output:



Medium Exercises

Exercise Q08 — Clamped Keyboard Movement [Medium]

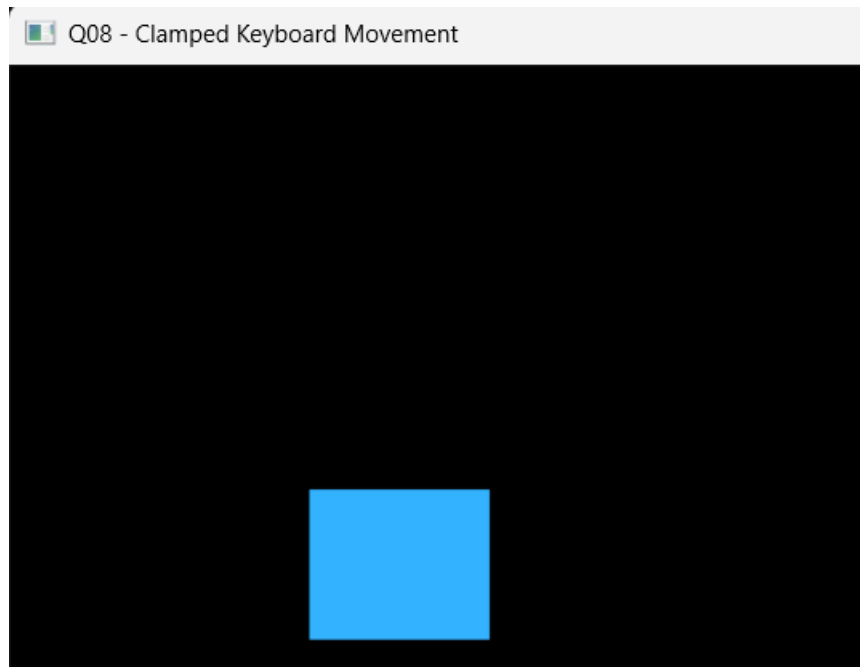
Objective: Practice keyboard-driven movement with boundary clamping.

Task: Use 'a' and 'd' keys to move a square left/right. Clamp its position so it can never move past the window edges (roughly -0.9 to 0.9).

Hints:

- After updating the position, check if it has gone past the limit and, if so, snap it back to the limit.
- Remember the square has a half-width — clamp based on its EDGE, not just its center.

Output:



Exercise Q09 — Active Motion Coordinate Display [Medium]

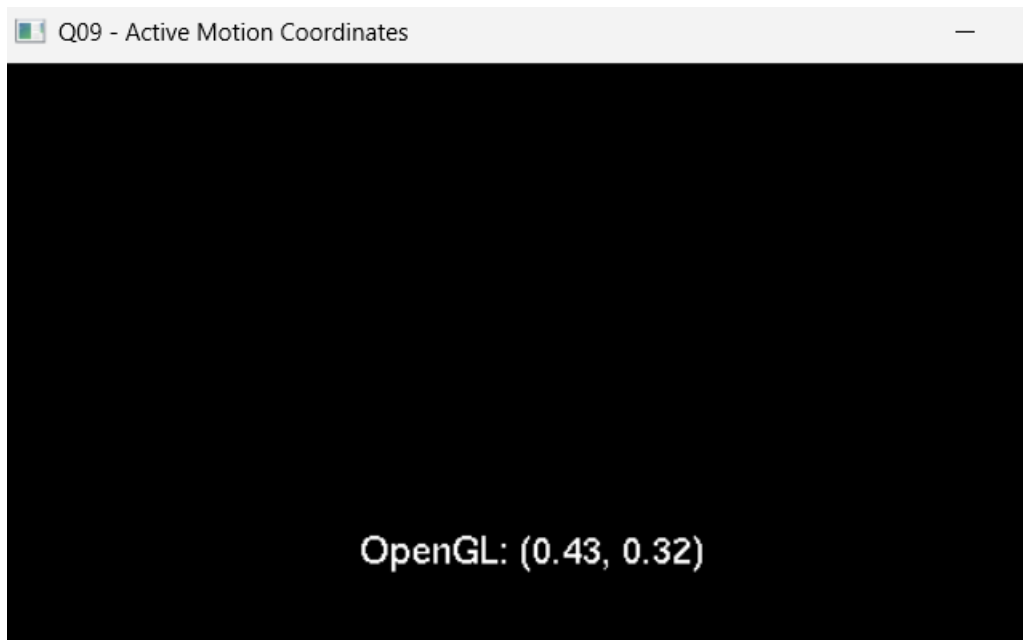
Objective: Practice `glutMotionFunc` combined with pixel-to-OpenGL coordinate conversion.

Task: While dragging with a mouse button held down, display the current cursor position as OpenGL-space text (not raw pixel coordinates).

Hints:

- Use `glutGet(GLUT_WINDOW_WIDTH) / GLUT_WINDOW_HEIGHT` to get the current window size for the conversion.
- Don't forget to flip the Y axis in your conversion formula.

Output:



Exercise Q10 — Passive Motion Pixel Readout [Medium]

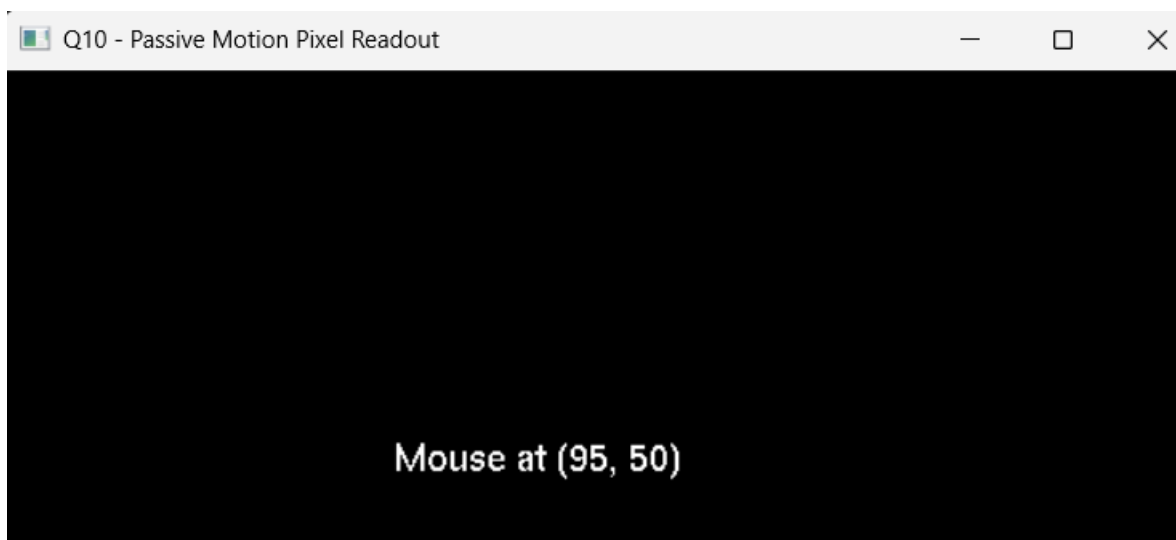
Objective: Practice `glutPassiveMotionFunc` for hover-only tracking.

Task: Without pressing any mouse button, moving the mouse over the window should display "Mouse at (x, y)" using raw PIXEL coordinates (no conversion needed this time).

Hints:

- `glutPassiveMotionFunc` gives you `x`, `y` directly in pixels — you can display them as-is.
- Use `snprintf` to build the display string each time the callback fires.

Output:



Exercise Q11 — Entry-Driven Background [Medium]

Objective: Practice `glutEntryFunc` for enter/leave detection.

Task: Use `glutEntryFunc` so the background turns light gray when the mouse pointer enters the window, and dark gray when it leaves.

Hints:

- Check state against `GLUT_ENTERED` and `GLUT_LEFT`.
- Update a background-color variable and call `glutPostRedisplay()` inside the callback.

Output:



Exercise Q12 — Idle-Driven Bouncing Label [Medium]

Objective: Practice `glutIdleFunc` for continuous animation.

Task: Use `glutIdleFunc` to continuously bounce a text label back and forth horizontally between the left and right edges of the window.

Hints:

- Store a small speed variable; reverse its sign when the label reaches either edge.
- Call `glutPostRedisplay()` every idle tick, or the animation will never actually appear.

Output:



Exercise Q13 — Timer-Driven Counter [Medium]

Objective: Practice a self-rescheduling `glutTimerFunc`.

Task: Use `glutTimerFunc` to increment and display an on-screen counter once every second, continuing indefinitely.

Hints:

- The timer callback must call `glutTimerFunc` again on ITSELF to keep repeating.
- Format the counter into a string with `sprintf` before passing it to `glutBitmapString`.

Output:



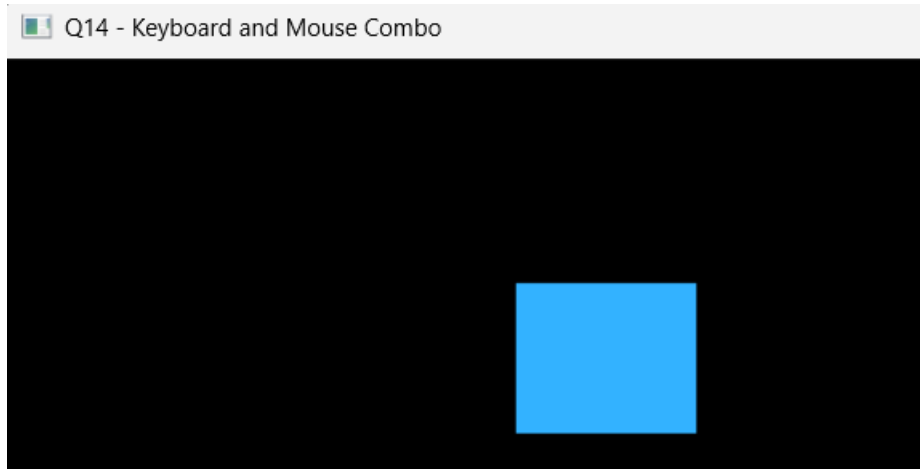
Exercise Q14 — Keyboard + Mouse Combo [Medium]

Objective: Practice combining two different callback types acting on the same state.

Task: Combine keyboard and mouse callbacks: keys 'w'/'s' move a shape up/down, and a mouse click resets it back to the center (0, 0).

Hints:

- Both callbacks should modify the SAME shared position variable(s).
- Both callbacks need their own glutPostRedisplay() call.

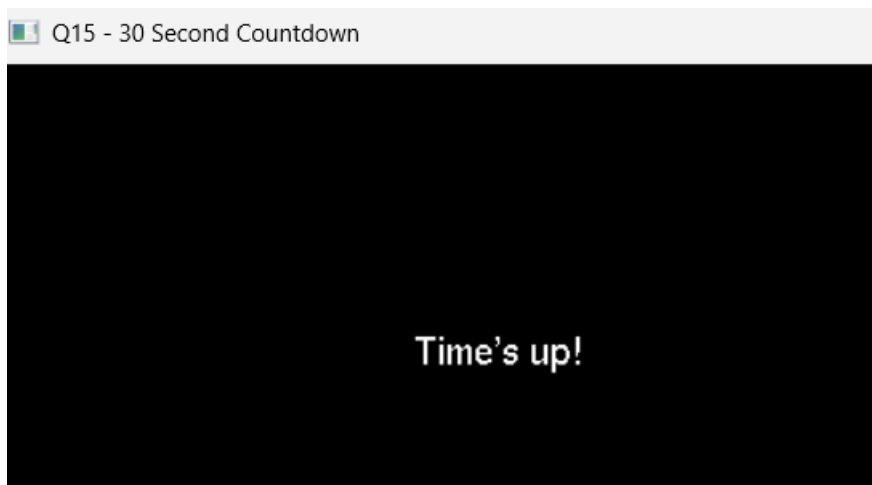
Output:**Hard Exercises****Exercise Q15 — 30-Second Countdown Timer [Hard]**

Objective: Practice a self-rescheduling timer with a terminating condition.

Task: Implement a 30-second countdown timer displayed as text. When it reaches 0, display "Time's up!" and STOP rescheduling the timer.

Hints:

- Only call glutTimerFunc again inside the callback if seconds remaining is still greater than 0.
- Use a boolean flag to switch the displayed message once time runs out.

Output:

Exercise Q16 — Click-and-Drag Square [Hard]

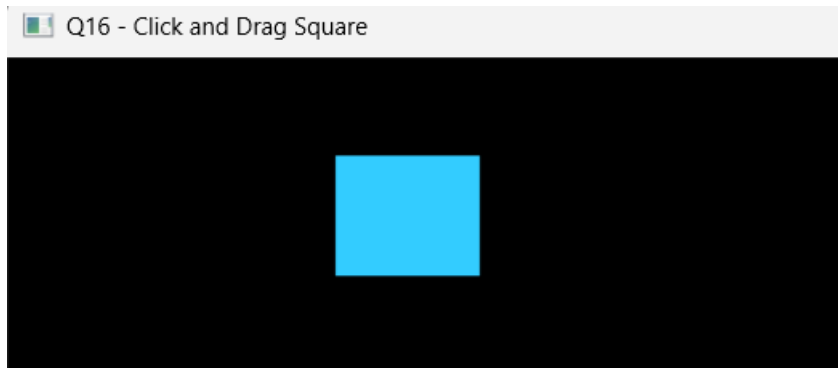
Objective: Practice tracking a 'currently dragging' boolean state across two different callbacks.

Task: Click and hold the left mouse button, then drag to move a square so it follows the cursor — but only while the button is held down.

Hints:

- Use `glutMouseFunc` to set a `dragging` boolean true on `GLUT_DOWN` and false on `GLUT_UP`.
- In `glutMotionFunc`, only update the square's position if `dragging` is true.

Output:



Exercise Q17 — Mouse-Controlled Stopwatch [Hard]

Objective: Practice combining `glutMouseFunc` with a self-rescheduling `glutTimerFunc` and a running/paused state.

Task: Build a stopwatch: left click starts/resumes it, right click pauses it, and the elapsed seconds are shown as on-screen text.

Hints:

- Keep the timer always rescheduling itself, but only increment the elapsed counter while a `running` boolean is true.
- Left click sets `running = true`; right click sets `running = false`.

Output:



Exercise Q18 — Entry + Idle Freeze Combo [Hard]

Objective: Practice gating a `glutIdleFunc` animation using state set by `glutEntryFunc`.

Task: Combine `glutEntryFunc` and `glutIdleFunc` so a shape animates (e.g., orbits or moves) via the idle callback only while the mouse pointer is inside the window; it should freeze in place when the mouse leaves.

Hints:

- Use a shared boolean ``inside``, updated only inside `glutEntryFunc`.
- In the idle callback, only update the animation state and call `glutPostRedisplay()` when ``inside`` is true.

Output:



Exercise Q19 — HUD Score with Color Milestones [Hard]

Objective: Practice combining click counting, HUD-style text, and conditional color changes.

Task: Build a HUD score overlay: each left click adds 1 point, shown as "Score: N" text over a shape. Every multiple of 5 points, the shape's color should change.

Hints:

- Use $\text{score} / 5$ combined with the modulo operator to determine which color 'stage' you're in.
- Keep the score-update logic and the color-update logic in the same callback so they always stay in sync.

Output:



Exercise Q20 — Interactive Text Placer (Capstone) [Hard]

Objective: Practice integrating pixel-to-OpenGL coordinate conversion, mouse clicks, passive motion, and idle animation together.

Task: Clicking anywhere in the window should convert the pixel coordinates to OpenGL coordinates and move BOTH a shape and its text label to that point. Also display a live passive-motion coordinate readout, and use `glutIdleFunc` to make the shape gently pulse in size.

Hints:

- Write one shared `toGL(x, y, &ox, &oy)` helper function and reuse it in both the mouse and `passivemotion` callbacks.
- Keep the pulsing effect subtle (e.g. $\text{size} = \text{base} + \text{amplitude} * \sin(\text{pulseAngle})$) so the marker and label stay clearly readable.

Output:

